



华中科技大学

异常控制流

许向阳

xuxy@hust.edu.cn

计算机科学与技术学院





第7章 异常控制流

用户进程



正常控制流：按照程序设定好的流程，
一条一条的执行指令

程序计数器：**RIP**，是如何自动变化的？

- 取指：根据 RIP 取指令
- 译码：RIP 自动加上指令的长度
- 执行：数据传送、算术运算、位运算.....
转移指令（有无条件）、CALL、RET
直接转移（调用）与 **间接转移**（调用）

Q: 转移指令执行时再次改变RIP，RIP如何得到？

- 重复上述过程





课堂目标

- 掌握中断和异常的基本概念
- 掌握中断和异常发生后，CPU的响应过程
- 掌握中断描述表的概念

第7章 异常控制流



华中科技大学

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    int i, j;
    int r = 0;
    char msg[20];
    for (i=0; i<20000; i++) {
        r = 0;
        for (j = 0; j < 100000; j++)
            r = r + 1;
    }
    printf("finish for ... \n");
    scanf("%s", msg);
    printf("your input : %s \n", msg);
    return 0;
}
```

循环语句的执行时间较长

Q: 在出现finish for ...
之前，输入 123456，
运行结果结果是什么？

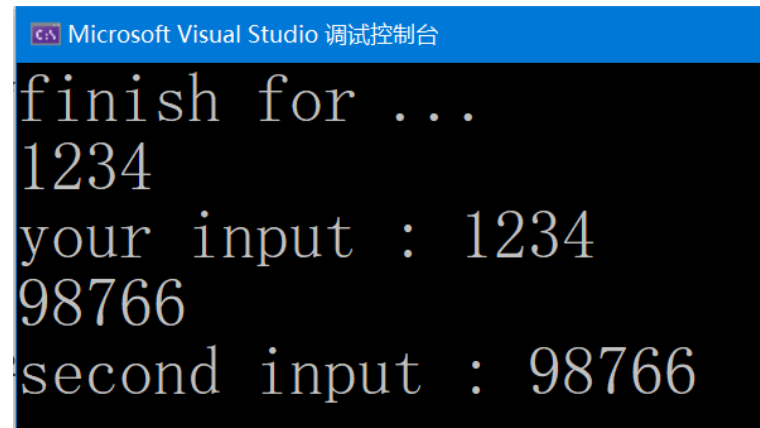




第7章 异常控制流

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    int i, j;
    int r = 0;
    char msg[20];
    for (i=0; i<20000; i++) {
        .....
    }
    printf("finish for ... \n");
    scanf("%s", msg);
    printf("your input : %s \n", msg);
    scanf("%s", msg);
    printf("second input : %s \n", msg);
    return 0;
}
```

Q: 在出现finish...之前,
输入一行 1234,
再输入一行 : 98766
运行结果结果是什么?



Microsoft Visual Studio 调试控制台

```
finish for ...
1234
your input : 1234
98766
second input : 98766
```

Q: 如何解释看到的现象?





第7章 异常控制流

```
#include <iostream>    #include <windows.h>
using namespace std;
#pragma comment(lib, "Winmm.lib")
#define delaytime 100
void CALLBACK TimeEvent(UINT uTimerID, UINT uMsg, DWORD_PTR dwUser,
DWORD_PTR dw1, DWORD_PTR dw2)
{    printf("\nHello\n"); }
int main()
{    int i = 0, timeID;
    timeID = timeSetEvent(delaytime, 10,
        (LPTIMECALLBACK)TimeEvent, 1, TIME_PERIODIC);
    char displaychar = 'A';
    for (i = 0; i < 5000; i++) {
        printf("%c", displaychar);
        displaychar++;
        if (displaychar == 'z' + 1) displaychar = 'A';
    }
    timeKillEvent(timeID);
    return 0;
}
```





第7章 异常控制流

运行结果片段

```
wxyzABCDEFGHIJKLMNOPQRSTUVWXYZ[\]_`abcdefghijklmnopqrstuvwxyz  
yzABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz  
ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrs  
Hello  
tuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstu  
vwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvw
```

```
defghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcde  
fghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ  
Hello  
STUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNQRST  
UVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyABCDEFGHIJKLMNQRSTU
```



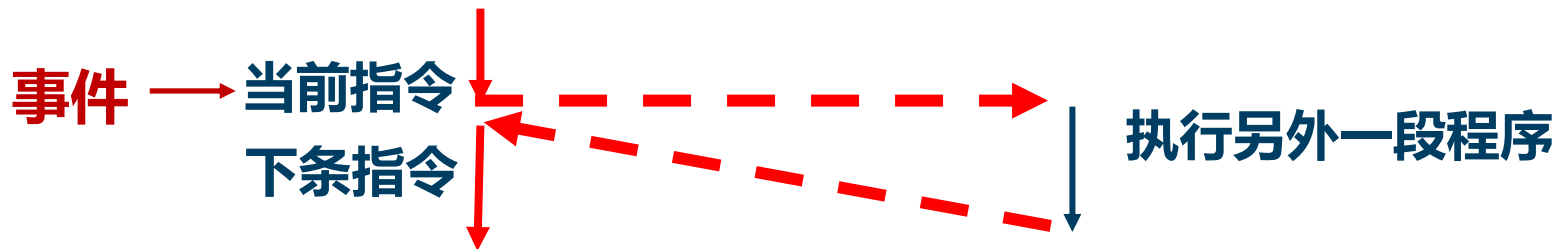
第7章 异常控制流

用户进程

正常控制流：按照程序设定好的流程，
一条一条的执行指令

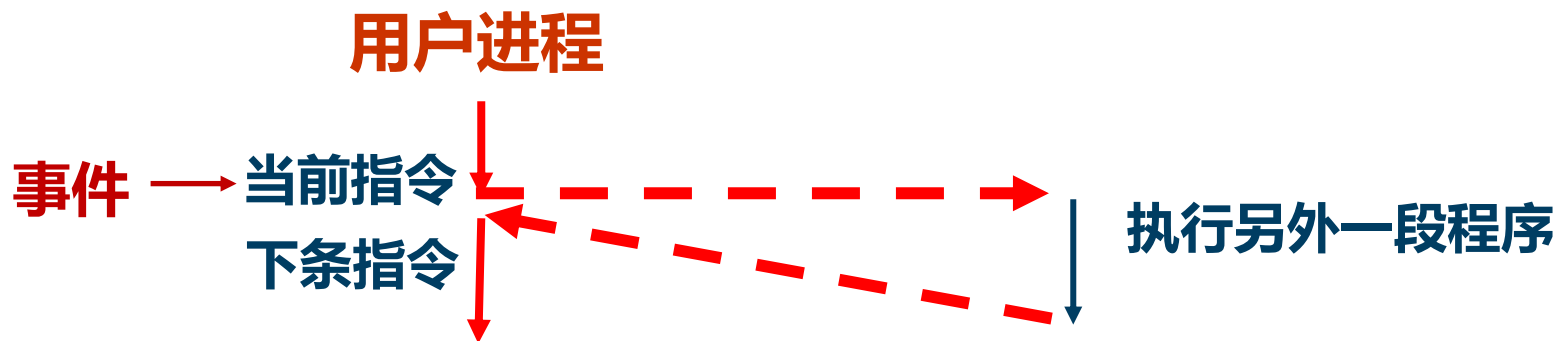
程序计数器： RIP 根据当前指令自动变化

用户进程



异常控制流：用户进程的正常控制流被打断；
插入了另外一段程序

第7章 异常控制流



- 会发生什么事件？事件来自于哪里？
- CPU 如何感知到事件的发生？感知发生了什么事情？
- 感知事件发生后，做什么处理？

保存现场——保存一些什么信息？

如何找到另一段程序的入口地址？

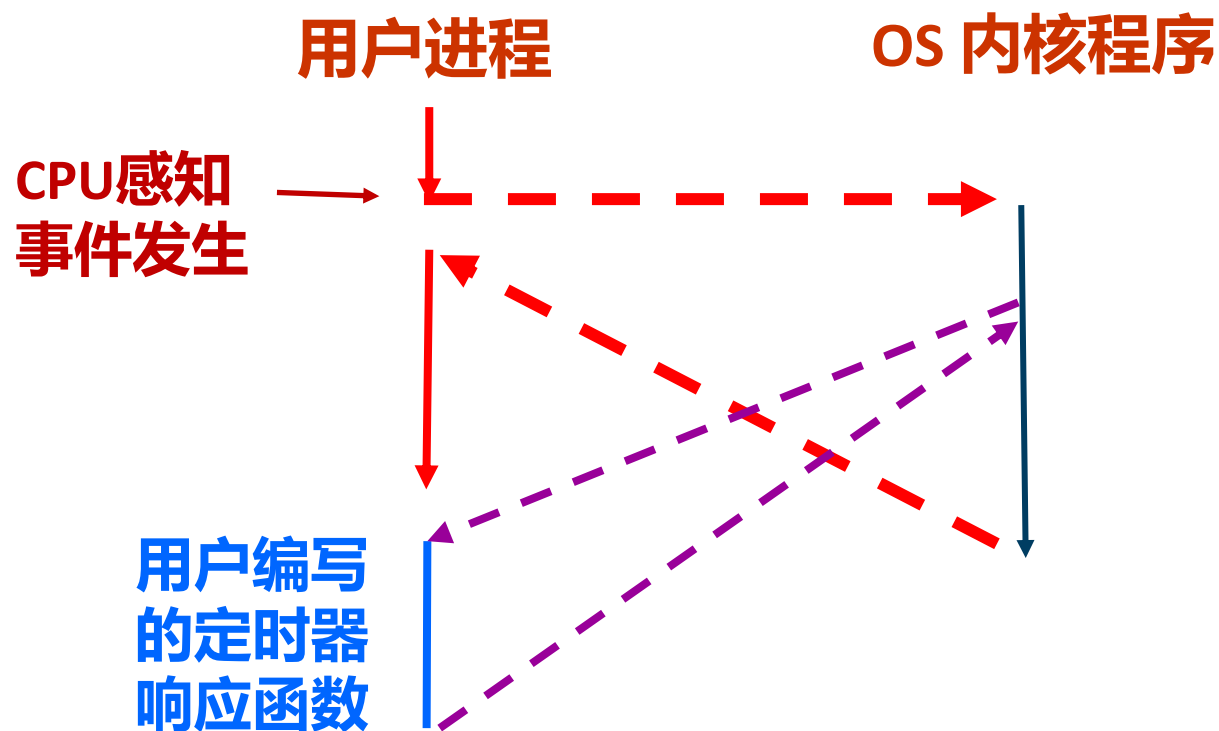
如何从另一段程序返回？

CPU、操作系统、用户程序 协同工作

第7章 异常控制流



华中科技大学



CPU、操作系统、用户程序 协同工作





第7章 异常控制流

1、中断与异常的基础知识

中断和异常的概念、中断描述符表

中断和异常的响应过程、软中断指令

2、Windows中的结构化异常处理

编写异常处理函数

异常处理程序的注册

全局异常处理程序的注册

3、C异常处理程序反汇编分析



中断与异常的基础知识



华中科技大学

中断和异常的概念

日常生活当中的“中断”

计算机世界中的“中断”

中断是CPU所具有的能打断当前执行的程序，转而为临时出现的事件服务，事后又能自动按要求恢复执行原来程序的一种功能。

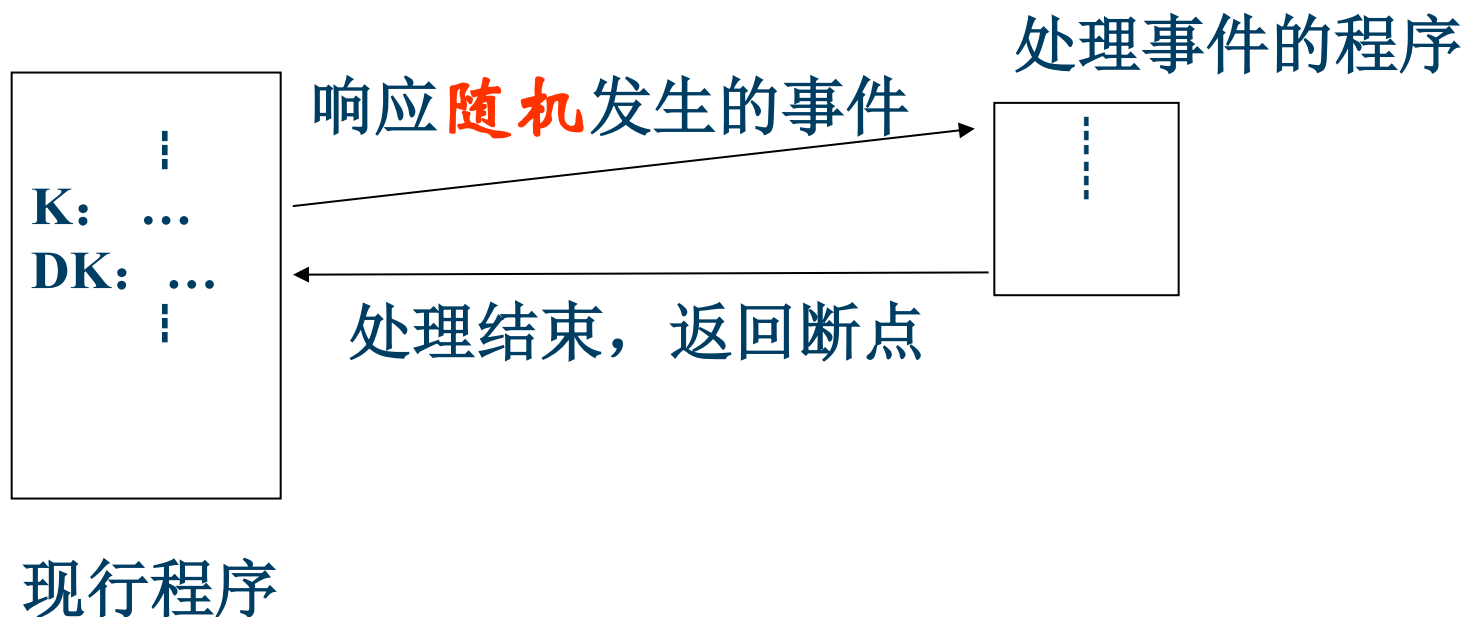


中断与异常的基础知识



华中科技大学

中断处理过程：



Q: 与子程序调用有什么差别？

事先安排好的 VS 随机发生的
有转移指令 VS 无转移指令



中断与异常的基础知识



华中科技大学

中断和异常的概念

- (1) 为什么要引入中断机制?
- (2) 有哪些事情会引起中断? **中断源**
- (3) CPU为什么能感知中断? **中断系统**
- (4) 在何处去找中断处理程序? **中断描述符表**
中断矢量表
- (5) 如何从中断处理程序返回?
- (6) 如何使用中断?



中断与异常的基础知识



华中科技大学

中断源分类

中断源

外部中断
(中断, 随机性)

不可屏蔽中断NMI:

电源掉电、存储器出错
或者总线奇偶校验错

可屏蔽中断INTR:

键盘、鼠标、时钟.....

开中断状态 (STI, IF=1)

关中断状态 (CLI, IF=0)

内部中断
(异常, 与CPU的状态和当前执行的指令有关)

CPU检测:

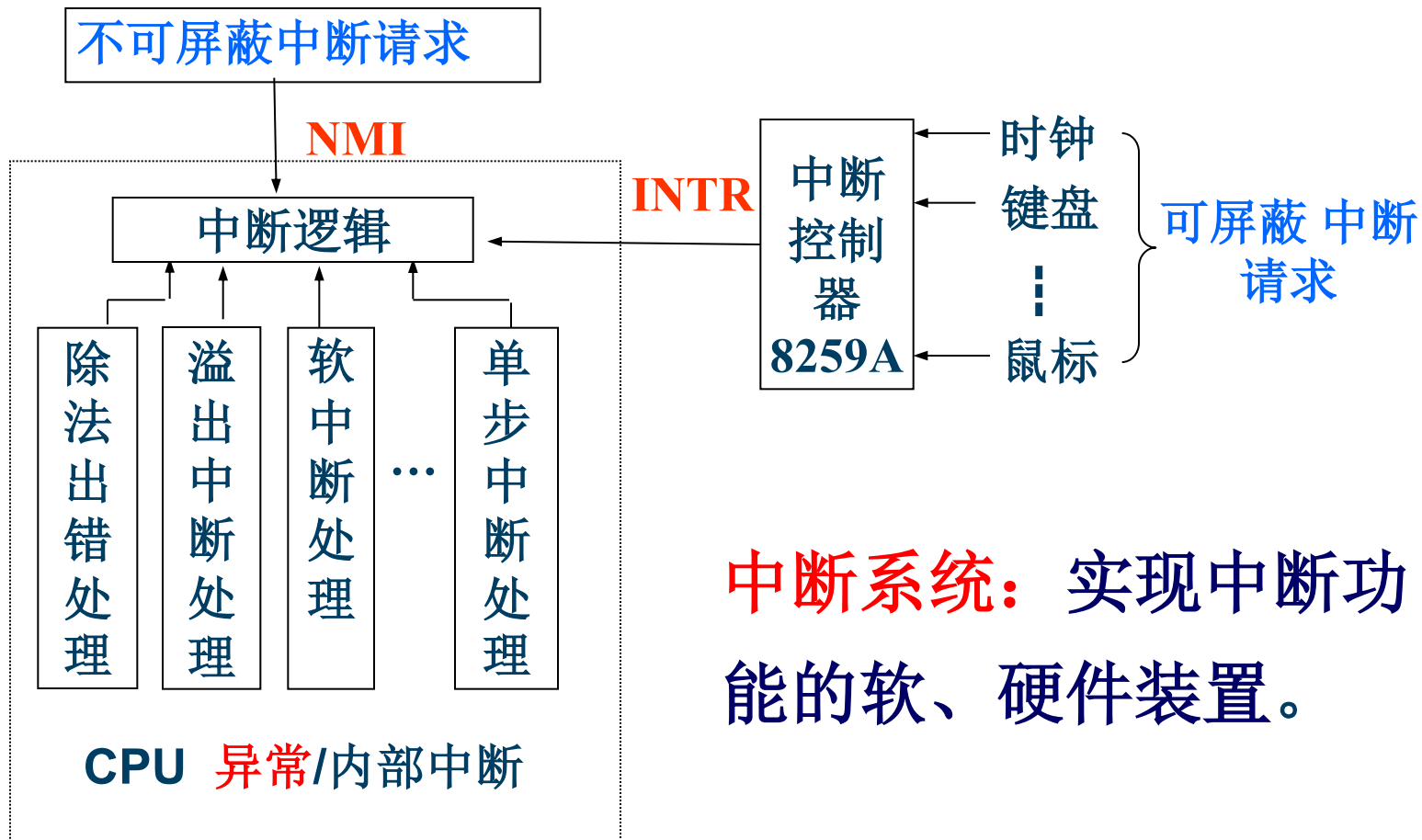
除法出错、单步中断、
协处理器段超越等。

程序检测:

软中断, 包括指令INTO、
INT n 和 BOUND等。



中断与异常的基础知识



中断系统：实现中断功能的软、硬件装置。



中断与异常的基础知识

优先级

中断/异常类型	优先级
除调试故障以外的异常 异常指令INTO、INT n、INT 3 对当前指令的调试异常 对下条指令的调试异常 NMI INTR	最高 ↓ 最低



中断与异常的基础知识

Intel 8086/8088 称外中断、内中断
从 80286 开始，称为中断、异常

- **中断**：由外部设备触发、与正在执行的指令无关、异步事件
- **异常**：与正在执行的指令相关的 **同步事件**。
CPU内部出现的中断，也称为**同步中断**。

一条指令的执行过程中，CPU检测到了某种预先定义条件，产生的一个异常信号，进而调用**异常处理程序**对该异常进行处理。





中断与异常的基础知识

- 在Intel CPU中，异常分为三类：
故障（faults）、陷阱（traps）、中止（aborts）。
- 在异常处理程序执行后，后续操作取决于异常的类型：
 - 重新执行引起异常的指令 —— 故障
 - 执行引起异常指令之下的指令 —— 陷阱
 - 终止程序运行 —— 中止
- 中断处理程序执行后会返回到被中断处继续执行



中断与异常的基础知识

➤ 异常 —— 故障

- 故障异常是在引起异常的指令之前或者指令执行期间，在检测到故障或者预先定义的条件不能满足时产生。
- 常见的故障异常
 - 除法出错（除数为0；除数很小被除数很大，商溢出）
 - 数据访问越界（访问一个不准本程序访问的内存单元）
 - 缺页
- 故障异常通常可以纠正，处理完异常时，引起故障的指令被重新执行。





中断与异常的基础知识

➤ 异常 —— 陷阱

- 在执行引起异常的指令之后，把异常情况通知给系统。
- 执行异常处理程序后，回到产生异常信号指令之下的一条语句。
- **软中断**是一种常见的**陷阱**。在程序中写了中断指令，执行该语句就会去调用中断处理程序，中断处理完后又继续运行下面的程序。
- 软中断调用与调用一般的子程序非常类似。借助于中断处理这一模式，可以调用操作系统提供的服务程序。
- **单步异常**，用于**一步步跟踪程序**。【标志寄存器中有一位 TF，TF=1时，CPU执行完一条指令就产生一个异常信号；TF=0时，则不产生异常信号】



中断与异常的基础知识



华中科技大学

➤ 异常 ——中止

- 中止是在系统出现严重问题时通知系统的一种异常；
- 引起中止的指令是无法确定的；
- 产生中止时，正在执行的程序不能被恢复执行。系统接收中止信号后，处理程序要重新建立各种系统表格，并可能重新启动操作系统；
- 中止的例子包括硬件故障和系统表中出现非法值或不一致的值；





中断描述符表

- 每一个中断或异常处理程序都有一个入口地址;
- 将中断和异常处理程序的入口地址等信息称为门 (gate), 就像一栋楼房的门代表了该楼房的入口一样;
- 根据中断和异常处理程序的类别, 将与之连接的中断描述符划分为三种门:
 - 任务门 (执行中断处理程序时将发生任务转移)
 - 中断门 (主要用于处理外部中断)
 - 陷阱门 (主要用于处理异常)
- CPU根据门提供的信息 (由IDT中的门属性字节提供) 进行切换, 对不同的门, 处理过程是有些差异的。





中断描述符表

中断号	名称	类型	相关指令	DOS下名称
0	除法出错	异常	DIV, IDIV	除法出错
1	调试异常	异常	任何指令	单步
2	非屏蔽中断	中断	-	非屏蔽中断
3	断点	异常	INT 3	断点
4	溢出	异常	INTO	溢出
5	边界检查	异常	BOUND	打印屏幕
6	非法操作码	异常	非法指令编码或操作数	保留
7	协处理器无效	异常	浮点指令或 WAIT	保留

8	双重故障	异常	任何指令	时钟中断
9	协处理器段超越	异常	访问存储器的浮点指令	键盘中断
0DH	通用保护异常	异常	任何访问存储器的指令 任何特权指令	硬盘（并行口） 中断
10H	协处理器出错	异常	浮点指令或WAIT	显示器驱动程序
13H	保留			软盘驱动程序
14H	保留			串口驱动程序
16H	保留			键盘驱动程序
17H	保留			打印驱动程序
19H	保留			系统自举程序
1AH	保留			时钟管理
1CH	保留			定时处理
20H~2FH	其它软/硬件 中断			DOS使用
0~0FFH	软中断	异常	INT n	软中断

中断描述符表



华中科技大学

中断类型码与对应的中断处理程序之间的连接表，存放的是中断处理程序的入口地址（也称为中断矢量或中断向量）。



中断描述符表

➤ 实方式下的中断矢量表

大小为1KB，起始位置固定地从物理地址0开始

主存	
00000H	- 类型0中断处理程序入口地址
⋮	- IP
00004H	- 类型1中断处理程序入口地址
⋮	- CS
00008H	- 类型2中断处理程序入口地址
⋮	⋮
003FCH	- 类型255中断处理程序入口地址
003FFH	-

中断号为1的中断处理程序的代码段

中断描述符表



华中科技大学

➤ 保护方式下的中断矢量表

- 在保护方式下，中断矢量表称作**中断描述符表** (IDT, Interrupt Descriptor Table)
- 按照统一的描述符风格定义其中的表项；
- 每个表项(称作**门**描述符)存放中断处理程序的入口地址以及类别、权限等信息，占8个字节，共占用2KB的主存空间。





中断描述符表

保护方式下的中断描述符表

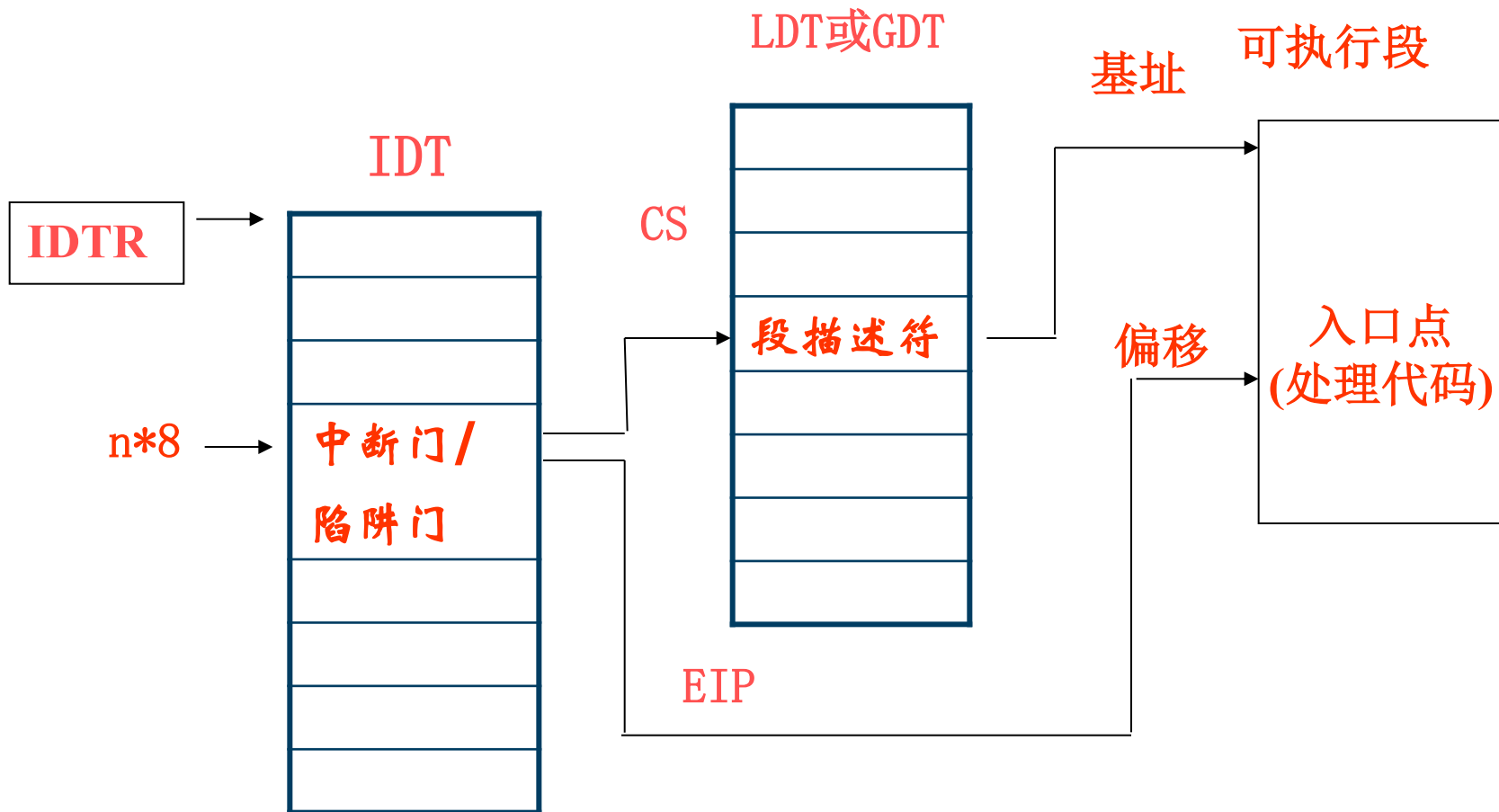
	主存	31	15	7	0
00000H	- 类型0中断处理程序入口信息	- 偏移值(高16位)	- 门属性	- 未用	
⋮					
00008H	- 类型1中断处理程序入口信息	- 段选择符(16位)	- 偏移值(低16位)		
⋮					
00010H	- 类型2中断处理程序入口信息				
⋮					
⋮					
007F8H	- 类型255中断处理程序入口信息				
007FFH					

IDTR决定IDT的起始PA。

IDT, Interrupt Descriptor Table



中断描述符表



从中断号到中断处理程序的转换过程



软中断指令

软中断通过程序中的**软中断指令**实现，所以又称它为**程序自中断**。

1. 软中断指令

格式: **INT** **n**

功能:

$(EFLAGS) \rightarrow \downarrow (ESP)$,

$0 \rightarrow TF$, 中断门还要将 $0 \rightarrow IF$





软中断指令

② (CS) 扩展成32位 → ↓ (ESP)

从门或TSS描述符中分离出的段选择符 → CS

③ (EIP) → ↓ (ESP)

从门或TSS描述符中分离出的偏移值 → EIP





软中断指令

2. 中断返回指令

格式: IRET

功能: ① $\uparrow (\text{ESP}) \rightarrow \text{EIP}$
② $\uparrow (\text{ESP})$ 取低16位 $\rightarrow \text{CS}$

恢复断
点地址

③ $\uparrow (\text{ESP}) \rightarrow \text{EFLAGS}$

恢复标志寄
存器的内容



实方式下中断处理程序示例

■ 上机演示

- ◆ 查看 INT 21H 中断处理程序的入口地址
内存的最低端，84H处
数据区 显示 0 : 84 处的内容
- ◆ 跟踪进入 中断处理程序 (Alt+F7)
查看 CS、IP是否为 上一步看到的程序入口地址
查看 堆栈是否存放的INT 21H 之下的断点地址





实方式下中断处理程序示例

- 新增一个中断处理程序
- 修改已有的中断处理程序以扩充其功能



实方式下中断处理程序示例

1. 新增一个中断处理程序的步骤

① 编制中断处理程序。

与子程序的编制方法类似，远过程，IRET。

② 为软中断找到一个空闲的中断号 m ；
或根据 硬件确定中断号。

③ 将新中断处理程序装入内存，将其入口地址
送入中断矢量表 $4*m - 4*m + 3$ 的四个字节中。





实方式下中断处理程序示例

1. 新增一个中断处理程序的步骤

Q: 如何得到新中断处理程序的入口地址?

方法一: SEG 子程序的名称
OFFSET 子程序的名称

方法二: INTP_ADDRESS DD 子程序的名称

方法三: 直接使用CS,当子程序与主程序在
同一个段时





实方式下中断处理程序示例

1. 新增一个中断处理程序的步骤

Q: 如何得到新中断处理程序的入口地址?

Q: 如何将该入口地址写入中断矢量表?

方法1 : 直接写中断矢量表的相关位置;

方法2 : DOS系统功能调用





实方式下中断处理程序示例

例：读出 08H 号中断处理程序的入口地址。

方法1：直接读中断矢量表的相关位置

```
MOV  AX, 0
MOV  DS, AX
MOV  AX, ds:[0020H]      ; 访问DS: [08H*4]单元
                           ; 即0: 0084H单元

MOV  BX, ds:[0034]
```

方法2：DOS系统功能调用

```
MOV  AX, 3508H
INT  21H
      (ES):(BX) 为 中断 08H 服务程序的入口地址
```





实方式下中断处理程序示例

1. 新增一个中断处理程序的步骤

Q: 如何得到新中断处理程序的入口地址?

Q: 如何将该入口地址写入中断矢量表?

Q: 如何调用新的中断处理程序呢?





实方式下中断处理程序示例

2. 修改已有中断处理程序以扩充其功能

- 编制程序段(根据扩充功能的要求, 应注意调用原来的中断处理程序)
- 将新编制的程序段装入内存;
- 用新编制程序段的入口地址取代中断矢量表中已有中断处理程序的入口地址。

Q: 如何调用老的中断处理程序呢?
直接使用 INT *, 行不行呢?**





实方式下中断处理程序示例

2. 修改已有中断处理程序以扩充其功能

要求:

- 扩充后的程序 调用原来的中断处理程序
即保留原有程序的功能
- 不能改原有的中断处理程序

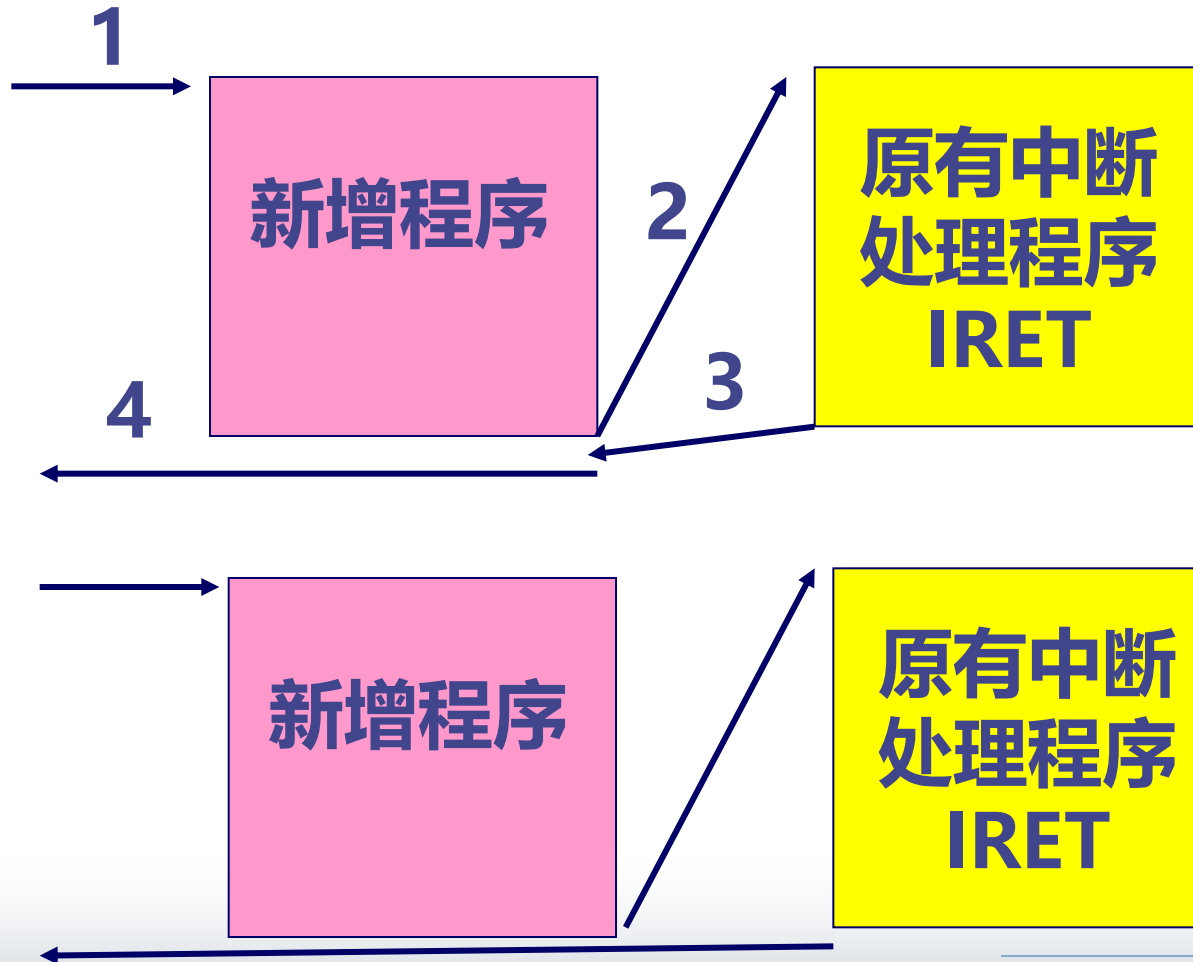
新增程序

原有中断
处理程序
IRET



实方式下中断处理程序示例

2. 修改已有中断处理程序以扩充其功能



实方式下中断处理程序示例

两种切换方法

中断响应 =》 新中断矢量

完成新增功能

方法1

方法2

```
PUSHF  
CALL DWORD PTR OLD_INT
```

```
JMP DWORD PTR OLD_INT
```

进入已有中断处理程序，完成原有功能（最后执行IRET返回到新增程序段）

进入已有中断处理程序，完成原有功能
（最后执行IRET退出中断处理程序）

IRET（真正退出中断处理程序）



实方式下中断处理程序示例

例: 编制时钟显示程序.

要求每隔1秒在屏幕的右上角显示时间。

(扩充原中断的功能)

**在该程序运行结束后，时间显示仍然继续。
在运行其它程序时，还看得到显示的时间。**





实方式下中断处理程序示例

程序涉及的知识要点分析:

- (1)如何知道是否到达1秒? 用什么中断合适?
- (2)如何取当前时间?
- (3)如何在指定位置显示时间?
- (4)如何在显示时间后 (改变了光标的位置) ,
不影响其他程序的运行?
- (5)如何在退出程序后, 仍能显示时间?





实方式下中断处理程序示例

程序涉及的知识要点分析:

(1)每隔1秒

定时中断，时钟中断

<PC中断大全 BIOS,DOS及第三方调用的程序
员参考资料>

<PC中断调用大全>

由8254系统定时器的0通道每秒产生18.2次，
该中断用于时钟更新。





实方式下中断处理程序示例

程序涉及的知识要点分析:

(1) 每隔1秒

引入一个变量，记录进入中断处理程序的次数。
当达到18次时，取时间，然后显示时间。

(2) 在屏幕的右上角显示时间

常用BIOS子程序,显示器驱动程序





实方式下中断处理程序示例

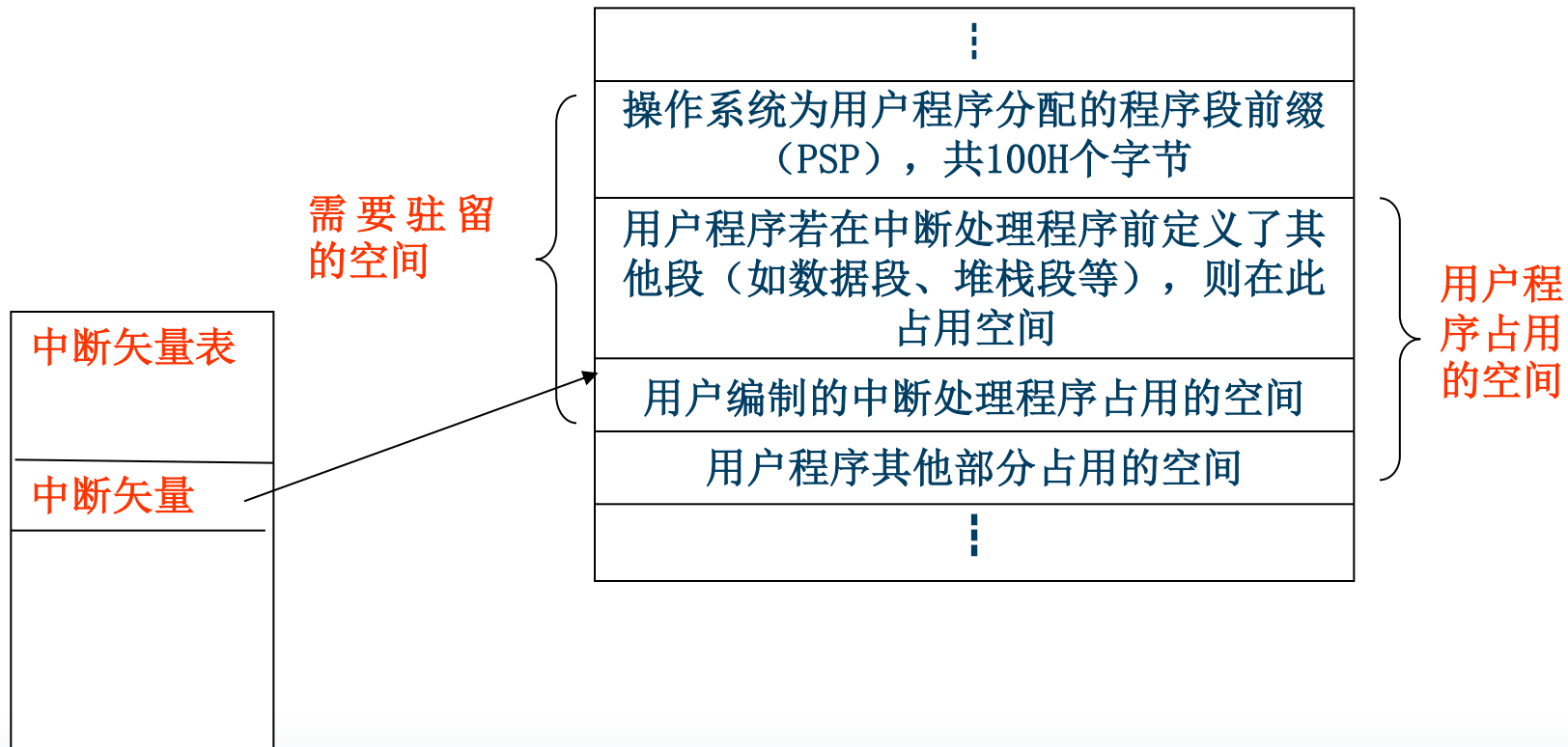
(3) 程序驻留

一个程序所占的主存储空间，在该程序运行结束后，不被回收。





实方式下中断处理程序示例





异常处理机制分类

Windows系统：结构化异常处理

Structured Exception Handling, SEH

Windows XP：向量化异常处理

Vectored Exception Handling, VEH

C++异常处理

C++ Exception Handler, C++EH





不同异常处理机制的比较

- Windows 的结构化异常处理和 C++ 异常处理是两种不同层次的错误处理机制；
- 两者目标相似（处理异常）
- 实现方式、适用场景和底层机制存在显著差异。



华中科技大学

不同异常处理机制的比较

1. 设计层次与适用范围

特性	结构化异常处理 (SEH)	C++ 异常处理
设计层次	操作系统级别 (Windows 特有)	编程语言级别 (C++ 标准)
适用场景	处理硬件异常 (如除零、内存访问违例) 和系统级异常	处理应用逻辑异常 (如无效参数、资源分配失败)
依赖关系	依赖 Windows 内核机制 (如 <code>EXCEPTION_RECORD</code>)	依赖编译器实现的异常模型 (与操作系统无关)





不同异常处理机制的比较

2. 语法与实现机制

特性	SEH	C++ 异常处理
语法结构	使用 Windows 特有关键字： <code>__try</code> 、 <code>__except</code> 、 <code>__finally</code>	使用 C++ 标准关键字： <code>try</code> 、 <code>catch</code> 、 <code>throw</code>
异常触发方式	由硬件或系统触发（如 <code>EXCEPTION_ACCESS_VIOLATION</code> ）	由开发者显式抛出（ <code>throw</code> ）
异常类型	基于错误代码（如 <code>DWORD</code> 类型的异常码）	基于对象（可抛出任意类型，推荐继承自 <code>std::exception</code> ）
异常过滤器	支持 <code>__except</code> 过滤器表达式（决定是否处理异常）	无过滤器，直接通过 <code>catch</code> 类型匹配
资源清理	<code>__finally</code> 块确保资源释放（无论是否发生异常）	依赖析构函数（RAII）自动清理资源



不同异常处理机制的比较

3. 性能与底层机制

特性	SEH	C++ 异常处理
性能开销	较低（直接由操作系统处理，无需类型匹配）	较高（涉及类型匹配和对象拷贝）
底层实现	基于线程的异常处理链 (<code>EXCEPTION_REGISTRATION_RECORD</code>)	基于编译器生成的异常表（如 <code>.pdata</code> 和 <code>.xdata</code> ）
跨平台性	仅限 Windows	符合 C++ 标准的编译器均支持





不同异常处理机制的比较

特性	向量化异常处理 (VEH)	结构化异常处理 (SEH)
作用范围	全局：影响整个进程，所有线程共享。	线程局部：基于线程栈帧，每个线程独立维护处理链。
注册方式	动态注册（通过 <code>AddVectoredExceptionHandler</code> API）。	静态编译时注册（通过 <code>__try / __except</code> 关键字）。
优先级	较低：在 SEH 处理链之后被调用。	最高：优先处理异常，若 SEH 未处理才会触发 VEH。
灵活性	更灵活，可注册多个处理函数，且可动态增删。	受限于代码结构，处理函数与栈帧绑定，生命周期固定。
典型应用场景	全局监控（如调试工具、崩溃报告、内存保护机制）。	局部错误恢复（如资源释放、错误回滚）。
执行流程	处理函数可修改异常上下文，允许继续执行原指令。	通常选择处理异常并终止当前代码块，展开栈。





不同异常处理机制的比较

VEH与SEH异常处理机制对比

依赖的编程模型	不依赖语言特性，可通过 WinAPI 直接使用。	与 C/C++ 语言深度集成（通过编译器关键字和库支持）。
---------	--------------------------	-------------------------------

核心机制对比

1. SEH（结构化异常处理）

- 基于线程栈帧链式结构，每个 `__try` 块对应一个 `EXCEPTION_REGISTRATION_RECORD` 节点。
- 异常分发时，系统从当前栈帧开始向上遍历处理链，直到找到能处理异常的节点。
- 通常用于**局部错误恢复**（如释放资源后退出函数）。

2. VEH（向量化异常处理）

- 通过进程全局链表管理异常处理函数（`VectoredHandlerList`）。
- 异常分发时，VEH 处理函数在 SEH 链处理完毕后被调用。
- 适合**全局监控或拦截**异常（如记录崩溃信息、动态补丁内存访问）。



不同异常处理机制的交互

- SEH 捕获 C++ 异常
- 在 Windows 上, C++ 异常默认通过 SEH 实现 (编译器将 throw 映射为 SEH 异常码 0xE06D7363)。
- 可通过 __try/__except 捕获 C++ 异常, 但需解析异常信息

```
int main()
{
    __try {        throw std::runtime_error("Error"); }
    __except (GetExceptionCode() == 0xE06D7363 ?
EXCEPTION_EXECUTE_HANDLER : EXCEPTION_CONTINUE_SEARCH)
    {    cout << "processing exception" << endl; }
    cout << "game over" << endl;
}
```





不同异常处理机制的交互

- C++ 捕获 SEH 异常
- 需启用编译器选项（如 MSVC 的 /EHa），使 catch(...) 能捕获所有异常（包括硬件异常）

// 属性 -> c/c++ -> 所有选项 -> 启用 C++ 异常

// 使 catch (...) 能捕获所有异常（包括硬件异常）

```
int main()
{
    try {        int* p = nullptr;        *p = 30; }
    catch(...) {
        cout << "processing exception" << endl;
    }
    cout << "game over" << endl;
}
```





结构化异常处理

```
#include <windows.h>
#include <iostream>
int main() {
    __try {        int* p = nullptr;
                   *p = 42; // 触发访问违例
    }
    __except (GetExceptionCode() ==
              EXCEPTION_ACCESS_VIOLATION
              ? EXCEPTION_EXECUTE_HANDLER
              : EXCEPTION_CONTINUE_SEARCH) {
        std::cout << "SEH 捕获到内存访问违例! " << std::endl;
    }
    return 0;
}
```





Windows 中的结构化异常处理

- 如何感知到出现了异常？
- 异常发生后如何处理？
- 如何去找到异常处理程序？
- 如何编写异常处理程序？
- 如何让系统找到自己的编写的异常处理程序？
- 异常处理程序执行后，再做什么？



Windows 中的结构化异常处理

```
#include <signal.h>           #include <setjmp.h>
#include <stdio.h>           #include <Windows.h>
jmp_buf buf;
LONG WINAPI Exceptionhandler(EXCEPTION_POINTERS* ExceptionInfo)
{
    printf("error \n");
    longjmp(buf, 1);
}

int main()
{
    if (!setjmp(buf)) {
        printf("then begin ... \n");
        Exceptionhandler(NULL);
        printf("then over ... \n");
    }
    else { printf("else branch ... \n"); }
    printf("finish .... \n");
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
then begin ...
error
else branch ...
finish ....
```



Windows 中的结构化异常处理

```
typedef struct __JUMP_BUFFER
{
    unsigned long Ebp;
    unsigned long Ebx;
    unsigned long Edi;
    unsigned long Esi;
    unsigned long Esp;
    unsigned long Eip;
    unsigned long Registration;
    unsigned long TryLevel;
    unsigned long Cookie;
    unsigned long UnwindFunc;
    unsigned long UnwindData[6];
} __JUMP_BUFFER;
```





Windows 中的结构化异常处理

▸ buf	0x00d6a138 {18
• buf[5],x	0x00d618f2
• eip,x	0x00d618f2
• buf[0],x	0x0113f954
• ebp,x	0x0113f954
• esp,x	0x0113f880
• buf[4],x	0x0113f87c
• eax,x	0x00000000

buf[0]: ebp

buf[4]: esp

buf[5]: eip

setjmp 保存现场;
返回eax为 0;

之上有 push 0

```

        if (!setjmp(buf)) {
00D618E6  push        0
00D618E8  push        offset buf (0D6A138h)
00D618ED  call        __setjmp3 (0D61253h)
00D618F2  add         esp, 8
00D618F5  test        eax, eax
00D618F7  jne         main+4Ch (0D6191Ch)
        printf("then begin ... \n");
    
```

```

        else { printf("else branch ... \n"); }
00D6191C  push        offset string "else branch ..."
00D61921  call        _printf (0D610CDh)
00D61926  add         esp, 4
    
```





Windows 中的结构化异常处理

buf	0x00d6a138 {18086228,
[0]	18086228
[1]	15519744
[2]	14028835
[3]	14028835
[4]	18086012
[5]	14031090
[6]	18086336
[7]	0
[8]	1447244336
[9]	0

Longjmp 跳转成 buf 的状态; 但 eax 置成了参数 1

```
longjmp(buf, 1);
00D617AE push 1
00D617B0 push offset buf (0D6A138h)
00D617B5 call _longjmp (0D61343h)
```

```
if (!setjmp(buf)) {
00D618E6 push 0
00D618E8 push offset buf (0D6A138h)
00D618ED call __setjmp3 (0D61253h)
00D618F2 add esp, 8 已用时间 <= 2ms
00D618F5 test eax, eax
00D618F7 jne main+4Ch (0D6191Ch)
printf("then begin ... \n");
```





Windows 中的结构化异常处理

```
jmp_buf buf;
LONG WINAPI Exceptionhandler(EXCEPTION_POINTERS* ExceptionInfo)
{
    printf("exception processing \n");
    longjmp(buf, 1);
}

int main()
{
    int a, t;
    SetUnhandledExceptionFilter(Exceptionhandler);
    if (!setjmp(buf)) {
        printf("then begin ... \n");
        a = 100;          t = 0;
        a = a / t;
        printf("then over ... \n");
    }
    else printf("else branch ... \n");
    printf("finish .... \n");
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
then begin ...
exception processing
else branch ...
finish ....
```



Windows 中的结构化异常处理

```
14 int main()
15 {   int a, t;
16     // SetUnhandledExceptionFilter(Exceptionhandler);
17     if (!setjmp(buf))
18     {   printf("then over ... \n");
19         a = 100;
20         t = 0;
21         a = a / t;
22         printf("then over ... \n");
    }
```

未经处理的异常

0x00531918 处有未经处理的异常(在 c_divide_zero.exe 中):
0xC0000094: Integer division by zero.

[复制详细信息](#) | [启动 Live Share 会话...](#)

▶ 异常设置

使用SetUnhandledExceptionFilter函数设置的是一个**筛选器**异常处理回调函数。





Windows 中的结构化异常处理机制

发生异常时，操作系统可以调用用户编写的异常处理函数。

- C++语言的关键字 **try ... catch** 只能捕捉/处理
用语句 **throw** 显式抛出的异常（不能捕捉其他异常）。
- VC++自己定义了另外的语句 **_try ... __except**，
用于捕捉/处理Windows所有的异常。
- **try...catch**和**_try..._except**的实现都是基于Windows **SEH**的
（Structured Exception Handling，结构化异常处理）：
- 按照**SEH**的规则，把异常处理程序加入到当前线程的**SEH**异常处理链中。





Windows 中的结构化异常处理

- `_try ..._except` 能够处理的所有的Windows异常
- `try ... catch` 只能捕捉用`throw`显式抛出的异常（常规编译）

```
try {  
    int *p = NULL;  
    *p = 0;  
}  
catch (...) {  
    printf("An exception.");  
}
```

~~//捕捉不到上面的异常~~

```
__try {  
    int *p = NULL;  
    *p = 0;  
}  
__except (exception filter) {  
    printf("An exception.");  
}
```

//可以捕捉到任何异常

注意：启用编译器选项（MSVC 的 `/EHa`）后，`catch(...)` 能捕获所有异常（包括硬件异常）





Windows 中的结构化异常处理

- 对于 `_try ..._except(exception filter)` 是一个表达式或者函数调用，表达式的值（函数调用的返回值）决定了处理异常后的下一步操作：

`EXCEPTION_EXECUTE_HANDLER` 已经处理异常，进程终止

`EXCEPTION_CONTINUE_EXECUTION` 已经处理异常，从产生异常的指令继续执行

`EXCEPTION_CONTINUE_SEARCH` 不处理，交给Windows处理





异常处理步骤

- (1) 首先交给调试程序进行处理（调试器优先级最高），如果不是调试状态则转(2)；
- (2) 在当前函数中的try-catch (`_try-__except`)中查找是否处理该异常，若是下列三种情况之一，则转(3)
 - ◆ 没有try-catch (`_try-__except`)
 - ◆ try-catch (`_try-__except`) 没有捕捉该异常
 - ◆ `__except`中表达式为 `EXCEPTION_CONTINUE_SEARCH`；
- (3) 依次调用当前进程的SEH处理链节点中的异常处理函数（Windows为每个线程建立一个SEH异常处理链），直到异常处理函数的返回值不是**ExceptionContinueSearch**。如果所有节点的异常处理函数都返回**ExceptionContinueSearch**，则转(4)；



异常处理步骤

- (4)系统再次检测进程是否正在被调试，如果被调试的话，则再一次通知调试器，如果调试器还是没有处理该异常，则转(5)；
- (5)系统检查是否有安装筛选器回调函数，如果有则调用它。如果没有筛选器回调函数或者筛选器回调函数的返回值为EXCEPTION_CONTINUE_SEARCH，系统则直接调用默认的异常处理程序终止进程（也就是崩溃的界面）；
（筛选器回调函数使用SetUnhandledExceptionFilter()安装）
- (6)如果SEH处理链没有处理当前异常（所有节点的异常处理函数都返回ExceptionContinueSearch），这时即使筛选器回调函数处理了当前异常，那么在终结之前系统会展开SEH链表，依次调用SEH链表中的回调函数（最后清理的机会）。





Windows 中的结构化异常处理

异常处理的编程方法:

根据Windows的异常处理机制，用户程序有3种方法去捕捉和处理异常：

- (1) C++规范的 `try...catch` 和 VC++的 `_try ..._except`。
- (2) 使用API函数 `SetUnhandledExceptionFilter` 去注册用于处理异常的筛选器回调函数。
- (2) 在SEH链表的表头插入一个新的结点（新的异常处理函数）。





Windows 中的结构化异常处理

方法1: try...catch、_try ..._except (1)

```
try {  
    int *p = NULL;  
    *p = 0;  
}  
catch (...) {  
    printf("An exception.");  
}
```

//捕捉不到上面的异常

```
try {  
    int *p = NULL;  
    if(p == NULL) throw -1;  
    else *p = 0;  
}  
catch (...) {  
    printf("An exception.");  
}
```

//使用**throw**显式抛出的异常才能够被**catch**捕捉到



Windows 中的结构化异常处理

方法1: try...catch、_try ..._except (2)

```
__try {  
    int *p = NULL;  
    *p = 0;  
}  
__except(EXCEPTION_EXECUTE_HANDLER) {  
    printf("An exception.");  
}
```

//能够捕捉到任何异常



Windows 中的结构化异常处理

方法2：筛选器回调函数

安装用于处理异常的筛选器回调函数：

LPTOP_LEVEL_EXCEPTION_FILTER WINAPI

SetUnhandledExceptionFilter(LPTOP_LEVEL_EXCEPTION_FILTER pTopLevelExceptionFilter)

说明：当一个异常没有被处理（**try**和**SEH**）且进程没有处于调试状态（不在**VS**或者别的调试器里运行），操作系统会调用注册的筛选器回调函数进行处理。

参数：**pTopLevelExceptionFilter** 是回调函数指针。**NULL**表示取消已经安装的筛选器回调函数。

返回：返回以前安装的回调函数指针。





Windows 中的结构化异常处理

筛选器回调函数:

回调函数原型:

**LONG WINAPI TopLevelExceptionFilter (
_In_PEXCEPTION_POINTERS ExceptionInfo)**

参数: **ExceptionInfo** 包含异常的详细信息 (异常代码、异常地址、异常参数等)。

返回: **EXCEPTION_EXECUTE_HANDLER (1)**
EXCEPTION_CONTINUE_EXECUTION (-1)
EXCEPTION_CONTINUE_SEARCH (0)





编写异常处理函数

回调函数：

当执行一条指令出现异常时，操作系统可以自动调用程序开发者编写的一个异常处理函数。

- 程序员编写出接口符合规范的异常处理函数；
- 向操作系统注册该函数；
- 回调函数可以显示错误信息、修复错误或者完成其他任务；
- 回调函数最后都要**返回一个值**来告诉操作系统**下一步做什么**。





编写异常处理函数

异常处理回调函数的格式:

```
EXCEPTION_DISPOSITION __cdecl _except_handler(  
    struct _EXCEPTION_RECORD *ExceptionRecord,  
    void * EstablisherFrame,  
    struct _CONTEXT *ContextRecord,  
    void * DispatcherContext);
```





编写异常处理函数

EXCEPTION_RECORD定义在winnt.h中，含有异常的编码、异常标志、异常发生的地址、参数个数、异常信息。

```
typedef struct _EXCEPTION_RECORD {  
    DWORD      ExceptionCode;  
    DWORD      ExceptionFlags;  
    struct _EXCEPTION_RECORD *ExceptionRecord;  
    PVOID      ExceptionAddress;  
    DWORD      NumberParameters;  
    ULONG_PTR  ExceptionInformation  
                [EXCEPTION_MAXIMUM_PARAMETERS];  
} EXCEPTION_RECORD;
```





编写异常处理函数

```
typedef enum _EXCEPTION_DISPOSITION {  
    ExceptionContinueExecution,  
        //已经处理了异常，回到异常触发点继续执行  
    ExceptionContinueSearch,  
        //继续遍历异常链表，寻找其他的异常处理方法  
    ExceptionNestedException,  
        // 在异常处理过程中再次触发异常  
    ExceptionCollidedUnwind  
        // 冲突 松开；发生了嵌套的展开操作  
} EXCEPTION_DISPOSITION;
```

下一步准备采取的动作





异常处理程序的注册

在windows系统中，对每一个进程都创建一个**线程信息块**TIB
(Thread Information Block) 来保存与线程有关的信息。
线程信息块的数据结构 **NT_TIB** 的定义在winnt.h中。

TIB 的第一个字段：

```
struct _EXCEPTION_REGISTRATION_RECORD *ExceptionList;
```

TIB 块在 fs:[0] 处



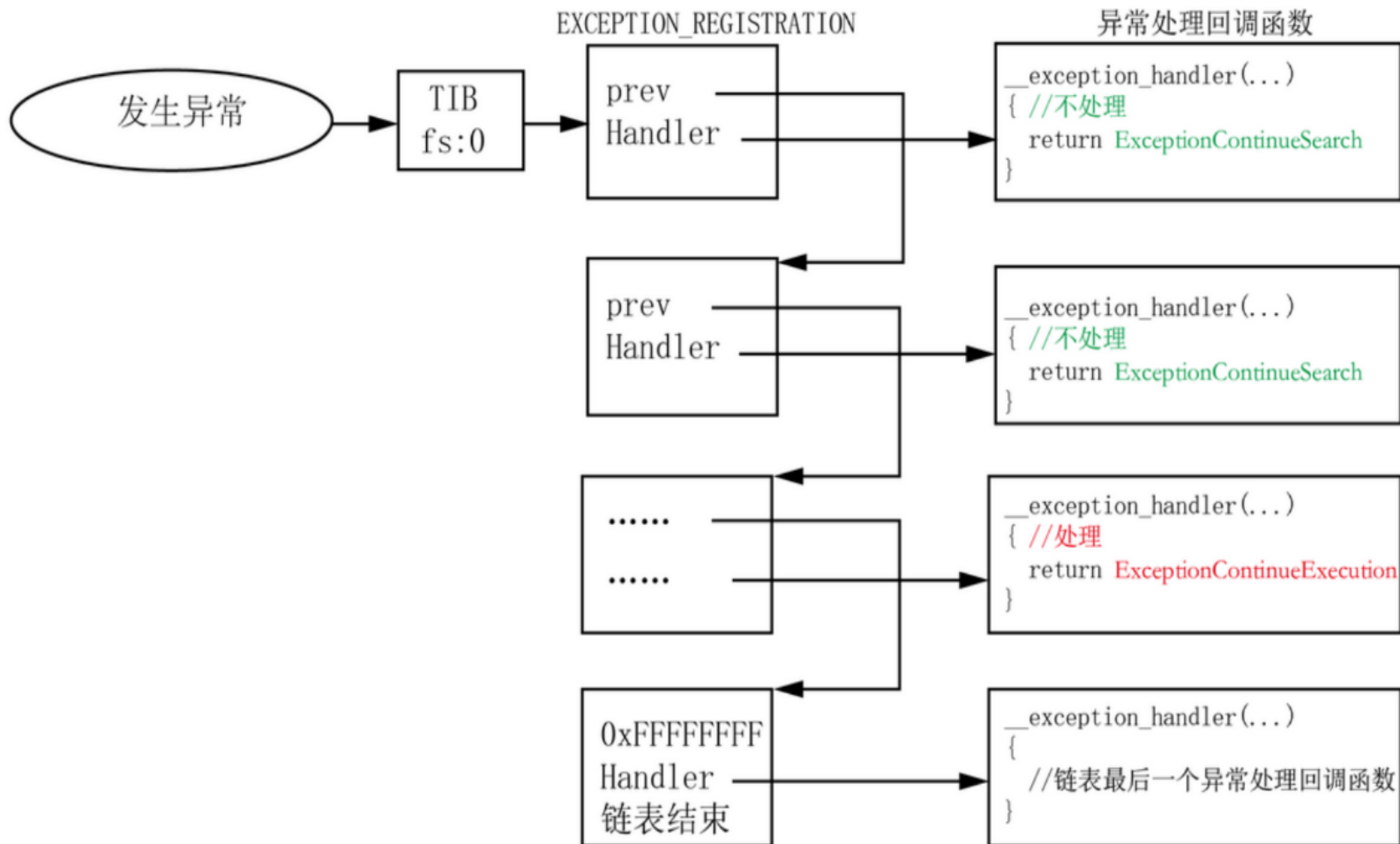


异常处理程序的注册

```
typedef struct _NT_TIB {  
    struct _EXCEPTION_REGISTRATION_RECORD *ExceptionList;  
    PVOID StackBase;  
    PVOID StackLimit;  
    PVOID SubSystemTib;  
#if defined(_MSC_EXTENSIONS)  
    union {  
        PVOID FiberData;  
        DWORD Version;  
    };  
#else  
    PVOID FiberData;  
#endif  
    PVOID ArbitraryUserPointer;  
    struct _NT_TIB *Self;  
} NT_TIB;
```



异常处理程序的注册





异常处理程序的注册

在程序执行过程中遇到异常事件时：

(1) 系统查看产生异常的进程是否正在被调试，若正在被调试，就向调试器发送EXCEPTION_DEBUG_EVENT事件。

(2) 如果进程没有被调试或者调试器不去处理这个异常，那么系统检查异常所处的线程，并在这个线程的环境中查看fs:[0]来确定是否安装有SEH异常处理回调函数，如果有的话则调用它。

(3) 回调函数尝试处理这个异常，如果可以修正错误，可将返回值设置为 ExceptionContinueExecution，此时系统将结束整个查找过程。





异常处理程序的注册

(4) 如果回调函数返回 **ExceptionContinueSearch**，告知系统它无法处理这个异常，那么系统将根据SEH链中的prev字段得到上一个回调函数地址并重复步骤（3），直到链中的某个回调函数返回**ExceptionContinueExecution**为止，查找结束。

(5) 如果到了SEH链的尾部却没有一个回调函数愿意处理这个异常，那么系统将再次检测进程是否正在被调试，如果被调试的话，则再一次通知调试器。





异常处理程序的注册

(6) 如果调试器还是不去处理这个异常或者进程没有被调试，那么系统检查有没有安装**筛选器回调函数**，如果有，则去调用它，筛选器回调函数返回时，系统默认的异常处理程序根据这个返回值做相应的动作。

筛选器回调函数使用SetUnhandledExceptionFilter安装

(7) 如果没有安装筛选器回调函数，系统直接调用默认的异常处理程序终止进程。





异常处理程序的注册

在程序执行过程中遇到异常事件时，操作系统中的函数 `RtlDispatchException` 会从 `FS:[0]` 指向的链表表头开始，依次调用每个节点指明的异常处理回调函数，直到某个异常处理回调函数的返回值为0为止（即函数的返回值为 `ExceptionContinueExecution`）。

异常处理回调函数的返回值为0，表示已经处理了该异常，该线程可以恢复执行。链表最末一项是操作系统在装入线程时设置的指向 `UnhandledExceptionFilter` 函数，该函数总是向用户显示 “Application error” 对话框。





异常处理程序的注册

如果程序在SEH链上挂了多个回调函数，并且每个回调函数都不处理异常的话，在系统默认的显示错误的对话框出现以后，每个回调函数都会被再调用一遍，这时参数中指定的异常代码是EXCEPTION_UNWIND，异常标志的取值是2，也就是EXCEPTION_UNWINDING标志。这种调用并不是要求回调函数去处理什么异常，而是告知回调函数：“你将要被卸掉了，自己处理一些后事吧”，在这时回调函数应该进行一些卸载前的扫尾工作并且返回ExceptionContinueSearch。





C异常处理程序反汇编分析

对C语言程序编译时，编译器会生成一些语句：

- 建立和恢复异常处理注册记录链表
- 使用FS:[0]指向链表表头
- 在异常处理所需信息的数据区 `__sehtable$_main` 中，会设置异常处理程序的入口地址。



补充知识

异常示例

EXCEPTION_INT_DIVIDE_BY_ZERO

整数除法的除数是 0

EXCEPTION_ACCESS_VIOLATION

企图从一个不具有权限的虚拟地址读取或者写入

EXCEPTION_STACK_OVERFLOW

栈溢出

EXCEPTION_ILLEGAL_INSTRUCTION

企图执行一个无效的指令

EXCEPTION_FLT_OVERFLOW

浮点数的指数超过所能表示的最大值

EXCEPTION_ARRAY_BOUNDS_EXCEEDED

企图越界访问数组元素，并且底层硬件支持边界检查

EXCEPTION_BREAKPOINT

断点被触发



异常编号示例 : winnt.h

```
#define STATUS_INTEGER_DIVIDE_BY_ZERO ((DWORD) 0xC0000094L)
```

```
#define STATUS_ACCESS_VIOLATION ((DWORD) 0xC0000005L)
```

```
#define STATUS_STACK_OVERFLOW ((DWORD) 0xC00000FDL)
```

```
#define STATUS_ILLEGAL_INSTRUCTION ((DWORD) 0xC000001DL)
```

异常编号示例 : minwinbase.h

```
#define EXCEPTION_INT_DIVIDE_BY_ZERO  
        STATUS_INTEGER_DIVIDE_BY_ZERO  
  
#define EXCEPTION_ACCESS_VIOLATION  
        STATUS_ACCESS_VIOLATION  
  
#define EXCEPTION_STACK_OVERFLOW  
        STATUS_STACK_OVERFLOW  
  
#define EXCEPTION_ILLEGAL_INSTRUCTION  
        STATUS_ILLEGAL_INSTRUCTION
```

- 中断和异常的概念

中断源、中断描述符表

- 中断和异常响应的过程

- 中断和异常处理程序的调用与返回

INT IRET

- 中断服务程序调用与一般的子程序调用的异同点